

Ett konkret exempel

Vi skall nu visa hur man kan typparametrisera vår klass `Stack` så att den accepterar vilken datatyp som helst:

```
template<class T>
class Stack {
public:
    // exceptions
    enum Exception { Underflow };

    // konstruktor
    Stack () : Top(0) { };
    ~Stack ();

    // metoder för stackmanipulation
    void push (const T & Data);
    T pop ();

private:
    // ett element i stacken
    struct Element {
        // data för detta element
        T Data;
        // nästa element under detta i stacken
        Element * Previous;
    };

    // stackens topp
    Element * Top;
};

// destruktör
template<class T>
Stack<T>::~~Stack () {
    // radera alla element
    try {
        while ( pop () );
    }
    catch ( Exception E ) {
        // stacken tom
    }
}

// push:a ett element på stacken
template<class T>
void Stack<T>::push (const T & Data) {
    // skapa nytt element
    Element * New = new Element;
    New->Data = Data;
    New->Previous = Top;

    // nytt toppelement
    Top = New;
}

// pop:a ett element från stacken
```

```

template<class T>
T Stack<T>::pop () {
    // är stacken tom?
    if ( Top == 0 ) {
        // stacken tom
        throw Underflow;
    }

    // spara data som är lagret i elementet
    T Data = Top->Data;

    // spara pekare till det element som blir nytt topp-element
    Element * Tmp = Top;
    Top = Top->Previous;

    // frigör minne och återvänd
    delete Tmp;
    return Data;
}

```

Man definierar en templateklass genom att placera `template<class T>` framför klassdeklarationen. Parametern `T` kan heta vad som helst, men ofta används `T` eller `Type`. Här är `T` en parameter som representerar den datatyp som stacken skall använda. Det är en helt *generisk typ*, d.v.s. den kan vara vilken datatyp som helst. Alla förekomster av `char` har ersatts med vår nya typ `T`. Samma `template<class T>` måste även placeras före metoddefinitionerna, samt även ett `<T>` mellan klassens namn och `::`. Det hela ser tyvärr ganska fult ut. Man kan om man vill skriva `template<class T>` på samma rad som själva metod- eller klasdefintionen, men detta sätt är det som förekommer mest. Datatypen `T` används som parameter, returvärde, lokala värden och som del i den `struct` som definierats. Man kan tolka `T` som en helt vanlig datatyp.

Instantiering av templateklasser

Nu har vi alltså definierat en första templateklass, nu är det dags att använda den också. Som man kunde gissa är även där syntaxen lite speciell. För att instantiera en `Stack` som fungerar på datatypen `int` kan vi använda följande sats:

```
Stack<int> IntStack;
```

Denna kan sedan användas efter deklarationen precis som en vanlig stack utan extra "påhäng". Om man vill skicka stacken som en parameter till en funktion el.dyl. måste man använda det fulla namnet `Stack<int>`. Vi kan även instantiera en strängstack och en floatstack:

```
Stack<string> S1;
Stack<float> Values;
```

Vikan göra ett enkelt program som använder sig av en stack av `int` enligt följande (det är en adaption av programmet i [Kapitel 12](#)):

```

int main () {
    Stack<int> S;
    char Choice = ' ';
    int Data;

    // iterera tills användaren vill avsluta
    while ( Choice != '3' ) {
        cout << "Välj: " << endl << "1 - push" << endl << "2 - pop" << endl;

```

```
cout << "3 - sluta " << endl << "-> ";

// läs in ett menyval
cin >> Choice;

// vad vill användaren göra
switch ( Choice ) {
case '1' :
    cout << "Elementets värde: ";
    cin >> Data;
    S.push ( Data );
    break;

case '2' :
    // försök poppa ett element
    try {
        Data = S.pop ();
        cout << "Poppade: " << Data << endl;
    }
    catch ( Stack<int>::Exception E ) {
        // stacken tom
        cout << "Stacken tom!" << endl;
    }
    break;
}
}
```

Notera att stacken är definierad som `Stack<int> S;` och att vi således måste även använda samma typdefinition då vi fångar en exception, alltså `Stack<int>::Exception E`.

Vi kan själv klart använda oss av olika instatieringar av vår `Stack`-klass i samma program, även samma funktion eller metod.

[Förutgående](#)
Typparametrisering

[Hem](#)
[Upp](#)

[Nästa](#)
Multipla parametrar